DAY 1

Module 3

Establishing Governance and Writing a Clarified Specification

Set the standard everything else gets checked against, then write against it.

What You'll Learn

1. Encode project or per-service governance in a constitution
2. Write a spec from both a feature request and a defect report using EARS-style requirements
3. Run a clarify pass to surface ambiguity before planning
4. Map the SDD lifecycle onto your team's existing SDLC

GOVERNANCE FIRST

The Constitution

Encodes principles, code quality standards, testing requirements, and architectural constraints.

Written once up front, informed by Module 2's discovery, and checked against by every later phase.

Without it, every generated plan and every piece of code is evaluated against nothing.

EARS-Style Requirements and Clarify

EARS format: "When/if [condition]… the system shall [response]." Testable, unambiguous, one behavior per statement.

Specs come from two different sources on this course: a new-feature request and an existing defect pulled from the tracker.

A clarify pass surfaces ambiguities and edge cases before they get baked into a plan.

SDLC MAPPING

The SDD lifecycle isn't a separate process: it maps onto the SDLC you already run.

Constitution and clarified spec → requirements and design.

Plan and tasks → technical design and iteration planning.

Checklist and analyze → QA and test planning. Implement → development. Review and living docs → code review, QA, and maintenance.

APPLYING THIS WITH

GitHub Spec Kit

SPEC KIT

Constitution, Specify, Clarify

`/speckit.constitution`	Creates or updates the constitution and keeps dependent templates in sync.
`/speckit.specify`	Creates the feature spec from natural language, focused on what and why, not tech stack.
`/speckit.clarify`	Asks up to five targeted questions about underspecified areas and encodes your answers back into spec.md. Run it as many times as needed.

APPLYING THIS WITH

OpenSpec

A partial fit: some of this module's concept maps, some doesn't.

OPENSPEC

Config Files, Deltas, and Explore

Governance gap: No dedicated governance command: project conventions live in openspec/config.yaml rather than a generated document.

`/opsx:propose`

Generates proposal.md plus delta specs/ marking ADDED, MODIFIED, REMOVED requirements, each with Given/When/Then scenarios: OpenSpec's equivalent of testable acceptance criteria.

Clarify gap: No single dedicated clarify command: ambiguity gets sharpened beforehand with /opsx:explore, or reconciled afterward with /opsx:update.

Module 3 at a Glance

	GitHub Spec Kit	OpenSpec
Governance	/speckit.constitution (dedicated command)	openspec/config.yaml (no generated document)
Spec writing	/speckit.specify	/opsx:propose (delta specs, Given/When/Then)
Clarify	/speckit.clarify (dedicated command)	/opsx:explore before, /opsx:update after

Key Takeaways

- Governance has to exist before code is judged against it.
- EARS keeps requirements testable regardless of which framework carries them.
- Clarify early: it's cheap now and expensive once a plan is built on top of ambiguity.

HANDS-ON NEXT

Write a constitution for your project, building on Module 2's discovery. Draft two specs against it: one for a new feature, one for the defect you pulled from the tracker. Run a clarify pass on both, then choose one to carry forward and set the other aside for Module 8.

Next: Module 4: From Clarified Spec to Reviewable Plan